43. Cloud SDK Patterns: Config, Retries, Pagination

43. Cloud SDK Patterns: Config, Retries, Pagination

Phase: Phase 6 — Systems and infrastructure
Estimated time: 1–2 days
Slug: `43-cloud-sdk-patterns`

Learning objectives

- Configure clients via env/role credentials
- Retry with exponential backoff + jitter
- Paginate list APIs correctly
- Treat cloud APIs as unreliable networks

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Cloud SDKs need retries, pagination, and config from env. Robust clients survive transient faults.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Retry transient errors with exponential backoff + jitter.
- Pagination: loop until next token empty.
- Config via env/flags; never hardcode secrets.
- Idempotent retries for safe verbs.
- Timeouts on every outbound call.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-cloud && cd /tmp/gw-cloud
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```go
import (
    "fmt"
    "math"
)

func Backoff(attempt int, baseMs float64) float64 {
    return baseMs * math.Pow(2, float64(attempt))
}

func main() {
    for i := 0; i < 4; i++ {
        fmt.Println(Backoff(i, 100))
    }
}
```

Run it:

```
go run .
```

Expected output:

```
100
200
400
800
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Implement backoff with max cap.
2. Paginate a fake NextToken API.
3. Read a config struct from env vars.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Retries

Retry transient errors (timeouts, 429, 503), not validation errors. Exponential backoff with jitter prevents synchronized stampedes.

```go
func Retry(ctx context.Context, fn func() error) error {
    // backoff loop checking ctx.Done()
}
```

Pagination

Most list APIs return a cursor/token. Loop until empty. Bound max pages.

Config

Prefer IAM roles / workload identity over long-lived keys in files. Separate config loading from business logic for tests.

Failure modes to watch while coding

- Infinite retries without deadline
- Retrying non-idempotent POSTs blindly
- Loading all pages into memory without need

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- tenacity ≈ custom backoff in Go.
- boto3 pagination ≈ token loops.
- Same twelve-factor config ideals.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Distinguish retryable (429/5xx) from permanent (400) errors.

Practice checklist

Before the exercise, confirm you can:

1. Explain "Cloud SDK Patterns: Config, Retries, Pagination" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Infinite retries without deadline
- Retrying non-idempotent POSTs blindly

- Loading all pages into memory without need

Exercises

Work in `exercises/43-cloud-sdk-patterns/`. Read that folder's `README.md` before you code.

What you will build

Implement retry/backoff helper per tests.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/43-cloud-sdk-patterns/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What is jitter?
2. Which errors retry?
3. Why cursor pagination?

Further reading

- https://aws.github.io/aws-sdk-go-v2/docs/